

Craigslist-like Web App Architecture

A complete architectural diagram and supporting notes for a classifieds web app (Craigslist-style).

1. Short description

A marketplace for user-posted classified ads: users can browse/search listings by category/location, post listings with images, message each other, save favorites, and report abuse. Frontend: **Angular**. Backend: **Java Spring Boot** (REST APIs + Spring Security). Primary DB: **PostgreSQL**. Search: **Elasticsearch** (or OpenSearch). File storage: **S3-compatible (AWS S3 / MinIO)**. Async messaging: **RabbitMQ / Kafka** (optional). CDN for static assets and image delivery.

2. High-level architecture (visual)

```
flowchart LR
    subgraph Client
        A[Angular SPA]
    end

    subgraph CDN
        CDNStatic[CDN - static assets & images]
    end

    subgraph LoadBalancer
        LB[Load Balancer (Nginx / ALB)]
    end

    subgraph AppLayer
        FE[Angular App Served or from CDN]
        API[Spring Boot REST API (stateless)]
        Auth[Auth Service (JWT / OAuth2)]
        Worker[Background Workers]
    end

    subgraph DataLayer
        PG[(Postgres - primary data)]
        ES[(Elasticsearch - search index)]
        S3[(S3 - images & attachments)]
        Cache[(Redis - cache, sessions, rate limit)]
        MQ[(RabbitMQ/Kafka - async tasks)]
    end
```

```
A -->|HTTPS| CDNStatic
A -->|HTTPS| LB
LB --> API
API --> PG
API --> ES
API --> S3
API --> Cache
API --> MQ
Worker --> MQ
Worker --> ES
Worker --> S3
Worker --> PG
CDNStatic --> S3
Auth --> API
API --> Auth
```

3. Components & responsibilities

Frontend (Angular)

- Single Page Application (SPA)
- Routes: Home, Category listing, Item detail, Create listing, Edit listing, User profile, Messages
- Handles client-side validation, SSR (optional with Angular Universal), and file upload to backend or direct S3 presigned URLs
- Uses JWT in Authorization header for protected API calls

Backend (Spring Boot)

- RESTful APIs (versioned, e.g., `/api/v1/`)
- Spring Security for authentication/authorization (JWT or OAuth2)
- Controllers → Services → Repositories (Spring Data JPA)
- Image upload endpoints or presigned URL generation
- Business logic: posting, editing, searching, messaging, reporting, moderation workflows
- Background workers for heavy tasks (image resizing, indexing, notifications)

Database (PostgreSQL)

- Core relational data: users, listings, categories, locations, messages, transactions, reports
- Use partitions or sharding strategies for large tables (listings)
- Read replicas for scaling read traffic

Search (Elasticsearch/OpenSearch)

- Full-text search, faceted filters (category, price range, location), geospatial queries (distance)
- Near-real-time indexing: write-through or background sync from DB via message queue

File storage (S3)

- Store images, thumbnails, attachments
- Use presigned URLs for direct upload from the browser to S3
- Generate thumbnails via background workers (Lambda or worker container)

Cache (Redis)

- Caching hot queries, user sessions, rate-limiting counters, ephemeral locks

Message queue (RabbitMQ / Kafka)

- Async tasks: indexing, sending emails/notifications, thumbnail generation, analytics events

CDN

- Serve static frontend bundle + user uploaded images via CDN for low latency
-

4. Data model (core entities)

- **User:** id, name, email, hashed_password, phone(optional), verified, created_at
 - **Listing:** id, user_id, title, description, price, currency, category_id, location(lat/lon or city_id), images[], status (active/closed/flagged), created_at, expires_at
 - **Category:** id, parent_id, name, slug
 - **Message:** id, from_user_id, to_user_id, listing_id, body, created_at, read
 - **Favorite:** id, user_id, listing_id, created_at
 - **Report:** id, reporter_id, listing_id, reason, status, created_at
-

5. Key APIs (examples)

```
GET /api/v1/listings?category=cars&lat=12.9&lon=77.5&radius=50&page=1
GET /api/v1/listings/{id}
POST /api/v1/listings (multipart/form-data or JSON + presigned uploads)
PUT /api/v1/listings/{id}
POST /api/v1/auth/login -> returns JWT
POST /api/v1/auth/register
POST /api/v1/messages
GET /api/v1/users/{id}/listings
POST /api/v1/report
```

6. Sequence for creating a listing (simplified)

1. User fills listing form in Angular and uploads images (client gets presigned URLs from API OR uploads through API)
 2. If presigned: client uploads images directly to S3; client sends listing metadata with S3 image keys to API.
 3. API stores listing in Postgres (status = pending)
 4. API publishes indexing event to MQ
 5. Worker consumes event -> generates thumbnails, writes to S3, updates listing images, indexes doc in Elasticsearch, marks listing active
 6. CDN serves images via S3 origin
-

7. Scaling & HA

- Deploy backend as stateless containers (Docker) behind LB; autoscale with Kubernetes (EKS/GKE) or ECS
 - Postgres: primary + read replicas; use connection pooling (PgBouncer)
 - Elasticsearch: multi-node cluster with replication
 - Redis: cluster mode for HA
 - S3: highly available managed storage
 - Use multi-AZ deployments for DB and cache
-

8. Security considerations

- Use HTTPS everywhere; HSTS
 - Secure JWT (short expiry, refresh tokens stored securely)
 - Rate limit APIs (especially create/listing/message endpoints)
 - Input validation and sanitization to prevent XSS/SQL injection
 - Virus/malware scan on uploads (optional)
 - Protect against scraping / bot detection (captcha on sensitive flows)
 - Role-based access control for admin/moderation
-

9. Observability

- Logging (structured logs): centralized ELK/EFK stack
 - Metrics: Prometheus + Grafana
 - Tracing: OpenTelemetry -> Jaeger
 - Alerting: on error rates, latency, resource utilization
-

10. CI/CD

- Pipeline: commit -> build -> unit tests -> integration tests -> build Docker image -> push registry -> deploy to staging -> smoke tests -> deploy to production
 - Use feature flags for gradual rollout
-

11. Deployment diagram (suggested)

- Kubernetes cluster with namespaces: `frontend`, `backend`, `workers`, `infra`
 - Managed services: RDS (Postgres), ElasticSearch (managed), S3, ElastiCache (Redis)
-

12. Next steps I can help with

- Create a detailed sequence diagram for user flows (post listing, search, message)
 - Draw a PNG/SVG of the architecture
 - Generate Spring Boot project skeleton + key endpoints
 - Generate Angular project skeleton + example pages
-

(If you'd like, I can convert the Mermaid diagram into a PNG or refine any particular area — architecture, data model, security, or a dev roadmap.)